MAPPING SHADOWS IN THE CLOUD : EXPLORING GOOGLE CLOUD ATTACK PATHS WITH GCP-HOUND AND OPENGRAPH

Faiz Karim
Senior Offensive Security Consultant

About me

Faiz Karim

Senior Offensive Security Consultant | Cyderes

India

6+ years of experience in cyberscurity, with hands-on expertise in both **red team** and **blue team** engagements.

Skilled in identifying, simulating, and defending against real-world attack scenarios.

Passionate about helping organizations strengthen their security posture through practical, adversary-driven insights.

Beyond Work

Love traveling and exploring new places.

Enthusiastic about trekking and outdoor adventures.

Agenda

01 Idea, Motivation & Credits

02 Building GCP-Hound: technical walkthrough

03 Enumerating and visualizing GCP attack paths

04 Tool limitations and future plans

05 Feedback / Q&A

Idea and motivation

- The project originated during preparation for a GCP cloud assessment for a client. A few days before the assessment, practice sessions in a CTF environment focused on GCP revealed a significant gap in available tooling.
- At that time, identifying and organizing attack paths for lateral movement required extensive manual work, and no tool comparable to AzureHound or SharpHound existed for GCP.
- Around the same time, John Hammond uploaded a video featuring Andy and Justin from SpecterOps, where they discussed the capabilities of BloodHound v8.0 and its compatibility with Open Graph.

Think Like an Attacker to Defend Better

Visualizing What You Can't See in the GCP Console -

- The GCP Console shows you IAM bindings as flat lists – it does not show you chained attack paths. GCP-Hound reveals things like:
  - A developer user has CanCreateKeys on the Compute Engine default service account, which has roles/editor.
  - The entire privilege escalation chain visualized as a graph in one view.

Identifying Misconfigurations Defenders Should Fix -

- roles/owner and roles/editor on service accounts are the most common misconfigurations in GCP projects.
- Orphaned or unused service account keys.

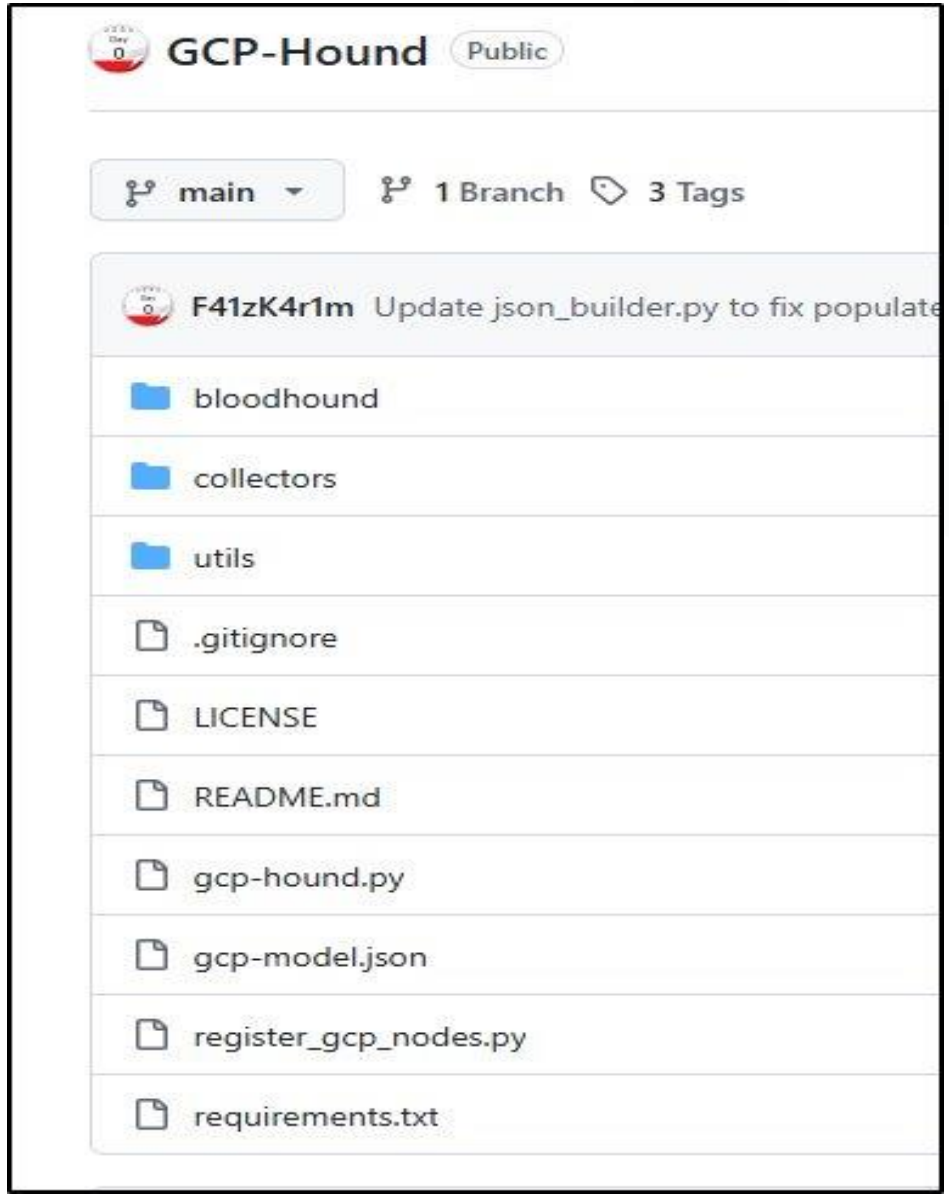
Principle of Least Privilege -

- Using GCP-Hound: "These 3 service accounts have roles/editor but only need roles/storage.objectViewer – here's exactly what attack paths disappear if we fix that."

Tool Structure

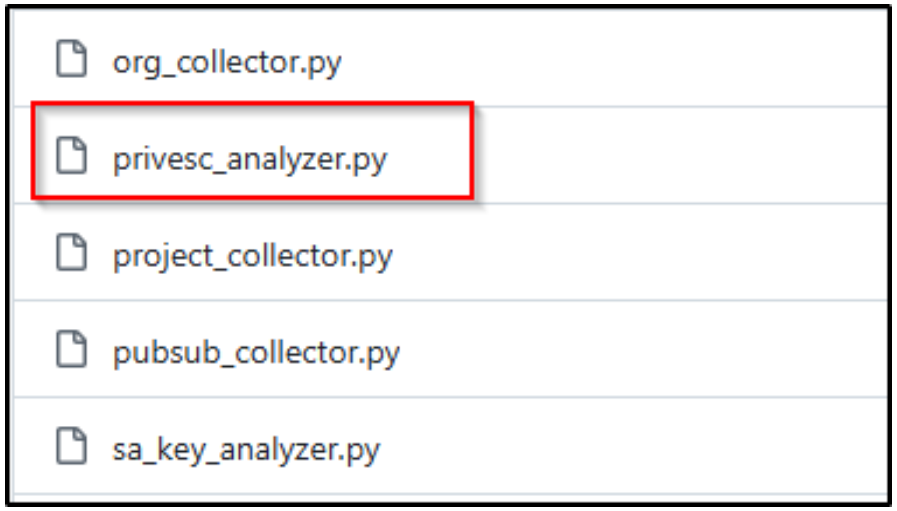
As of now GCP-Hound consist of 4 different parts:

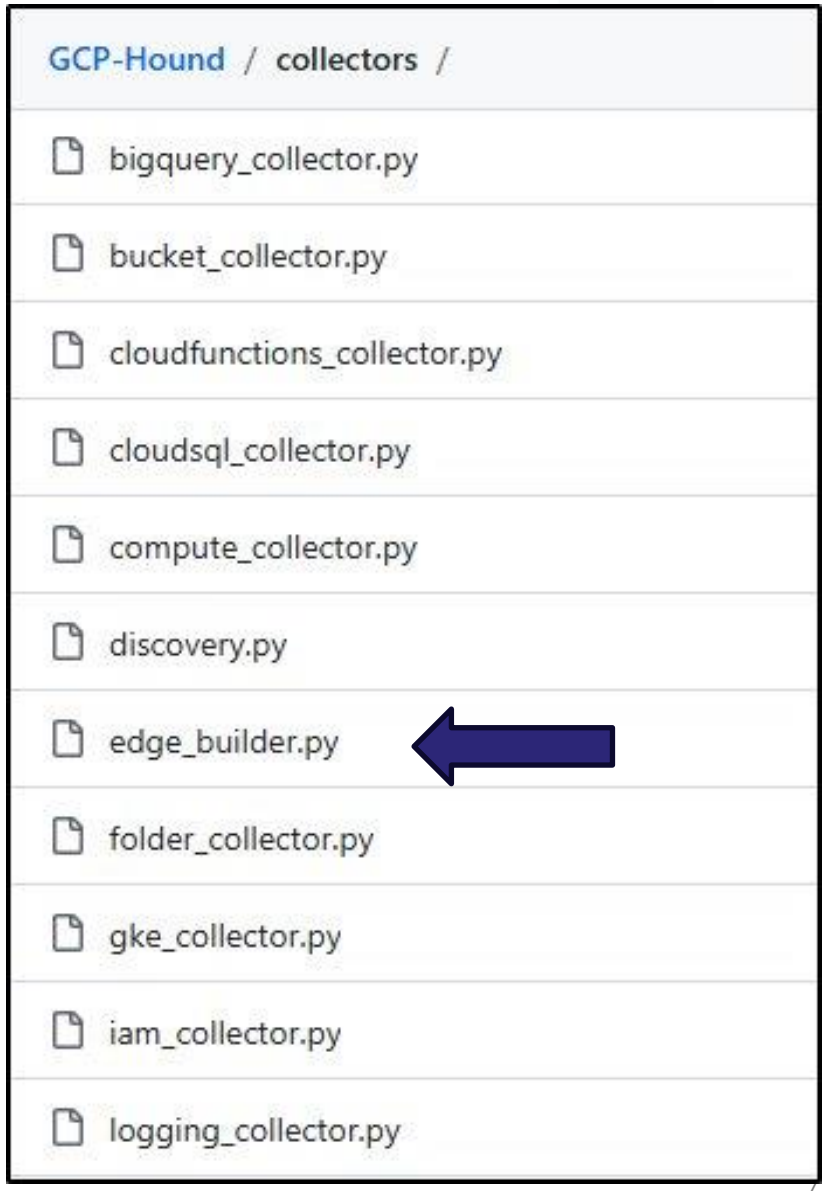
- gcp-hound.py (main file)
- Utils - which contains auth.py and id_utils.py (which normalizes and extracts all valid forms of dataset/resource IDs)
- json_builder.py (inside bloodhound directory)
- Collectors that contains multiple python scrips where each script gathers data for a specific GCP resource or privilege type.



Different collectors

- As of now total 16 collectors are being used to collect the data from different components of the GCP.
- Post collection, the edge_builder.py script just creates the edges based on the enumerations results.
- I have also added privesc_analyzer.py, which prioritize the privilege escalation vectors based on their nature.





Authentication

GCP-Hound supports two authentication methods:

- Most simple way is just to export the json file with the creds in env variable "GCP_CREDS".

```
└$ export GCP_CREDS=/opt/clients/gcloud/[redacted]nsdk.json
```

- USING gcloud cli to perform auth login with cred file or interactive logon.

```
└$ gcloud auth login --cred-file=/opt/clients/gcloud/[redacted]dk.json

You are already authenticated with 'fir[redacted]rviceaccount.com'.
Do you wish to proceed and overwrite existing credentials?

Do you want to continue (Y/n)?  y


Authenticated with service account credentials for: [fireb[redacted]am.gserviceacco
Your current project is [None].  You can change this setting by running:
  $ gcloud config set project PROJECT_ID
```

Phase 1: execution

Options to run the script:

- Python3 gcp-hound.py
- "-v" for verbose output
- "-d" for debugging mode
- "-p" to target only specific project
- "-i" with impersonation (if the current object has privilege)

```
└$ python3 gcp-hound.py -v

GCP-HOUND

🔐 Google Cloud Platform Security Assessment & Attack Surface Discovery
🎯 Comprehensive GCP Privilege Escalation Detection & BloodHound Integration

[*] Verbose mode: Enabled
[*] Output directory: ./output

[+] Running as: firebas[redacted]gserviceaccount.com

[*] 🔍 Phase 1-3: Reconnaissance & Resource Discovery
[+] Discovered 1 projects via Cloud Resource Manager API
    - [redacted]
[*] Discovered 1 projects using Cloud Resource Manager API
```

Phase 2: Available API detections and admin check

```
[*] Phase 2: API Capability Assessment
[+] Found 53 enabled APIs in [redacted]

[*] Enumeration Capabilities by Project:

    Project: [redacted]
      Service Accounts   √
      Storage Buckets    √
      Users/Groups       ✗
      Compute Instances  √
      Secrets            √
      BigQuery           √
      GKE Clusters       ✗
      Cloud Functions    √
      Logging           √

[+] Overall Enumeration Coverage: 8/9 collectors available
[*] Enabled capabilities: Service Accounts, Storage Buckets, Users/Groups, Compute Instances, Secrets, BigQuery, Cloud Functions, Logging
02:47:15 [WARNING] Encountered 403 Forbidden with reason "accessNotConfigured"

[*] Google Workspace Admin Status Check:
    ✗ NO ADMIN ACCESS
```

Phase 3: Resource Enumeration

```
[*] Phase 3: Resource Enumeration
[*] Starting service account enumeration with pagination ...
[+] Found 5 service accounts in [redacted]
[+] Service account enumeration complete: 5 accounts across 1 projects
[+] Found 5 service accounts
[*] Starting storage bucket enumeration ...
[+] Found 3 storage buckets in [redacted]
    - [redacted]age.app (ASIA-SOUTHEAST1, REGIONAL, public access: inherited)
    - [redacted]a (ASIA-SOUTHEAST1, STANDARD, public access: inherited)
    - [redacted] (US, STANDARD, public access: inherited)
[+] Storage bucket enumeration complete: 3 buckets across 1 projects
    - [redacted]rage.app (ASIA-SOUTHEAST1)
    - [redacted]a (ASIA-SOUTHEAST1)
    - [redacted] (US)

[*] ENUMERATING SECRET MANAGER SECRETS ...
[~] No secrets found in [redacted]

[*] ENUMERATING COMPUTE ENGINE INSTANCES ...
[~] No compute instances found in [redacted]

[*] ENUMERATING BIGQUERY DATASETS AND TABLES ...
[~] No BigQuery datasets found in [redacted]

[+] BIGQUERY ENUMERATION SUMMARY:
 📊 Total Datasets: 0
 📋 Total Tables: 0
 🚨 HIGH-Risk Datasets: 0
 ⚠ MEDIUM-Risk Datasets: 0
 🏗 Projects Processed: 1
```

SA Enumeration

Storage buckets

Secrets

Compute Engine

BQ Data sets & Table

```
[+] Found 3 log sinks, 2 log buckets, 0 log metrics

[*] Phase 3A: IAM Policy Enumeration
[*] IAM Collector: Analyzing 1 projects for IAM bindings ...
[+] Found 20 IAM bindings in project: [redacted]
[+] IAM Collector: Completed analysis of 1 projects with IAM data
[*] Phase 3A2: Service Account Permission Analysis
[*] Service Account Permissions: Analyzing 5 service accounts ...
[*] Analyzing 5 service accounts in project: [redacted]
[+] Service account [redacted]: 1 IAM bindings
[+] Service Account Permissions: Found policies on 1 service accounts

[*] Phase 3B: Organizational Structure Enumeration
[!] No organizations provided - discovering accessible folders
[*] Attempting to discover accessible folders via search ...
[+] Discovered 0 accessible folders via search
```

Log Sinks

IAM Policies

Phase 4: Privilege escalation analysis

In this phase we focusing upon how exactly we can leverage the current configuration for privilege escalation (vertical/horizontal).

```
🚨 COMPREHENSIVE PRIVILEGE ESCALATION ANALYSIS
[*] Testing 20 escalation methods across 1 projects...

[*] Analyzing project: [redacted]
    🚨 CRITICAL Can create keys for: [redacted]rvice account
    🚨 CRITICAL Can create keys for: [redacted]
    🚨 CRITICAL Can create keys for: [redacted]
    🚨 CRITICAL Can create keys for: [redacted]t service account
    🚨 CRITICAL Can create keys for: [redacted]
    🚨 CRITICAL Can generate access tokens for: A[redacted]ult service account
    🚨 CRITICAL Can generate access tokens for: f[redacted]dk
    🚨 CRITICAL Can generate access tokens for: f[redacted]access
```

CanCreateKeys

Phase 4: Privilege escalation analysis

The privilege escalation can be performed accessing the secret keys, by impersonation or in some condition by signing the blob or modifying IAM policies.

```
⚠ HIGH Can impersonate service account: [redacted] service account
⚠ HIGH Can impersonate service account: [redacted]sdk
⚠ HIGH Can impersonate service account: [redacted]-access
⚠ HIGH Can impersonate service account: [redacted] default service account
⚠ HIGH Can impersonate service account: [redacted]
```

CanImpersonate

```
⚠ HIGH Can sign blobs as: [redacted]lt service account
⚠ HIGH Can sign blobs as: [redacted]k
⚠ HIGH Can sign blobs as: [redacted]ccess
⚠ HIGH Can sign blobs as: [redacted]efault service account
⚠ HIGH Can sign blobs as: [redacted]
```

CanSignBlob

```
⚠ HIGH Can sign JWTs as: [redacted]lt service account
⚠ HIGH Can sign JWTs as: [redacted]k
⚠ HIGH Can sign JWTs as: [redacted]ccess
⚠ HIGH Can sign JWTs as: [redacted]efault service account
⚠ HIGH Can sign JWTs as: [redacted]
```

CanSignJWT

Phase 5: Attack path graph building

- Combines all collector output into a unified graph structure.
- Nodes: identities (users, SAs, groups), resources (instances, buckets, functions).
- Edges: IAM bindings, impersonation paths, resource access.

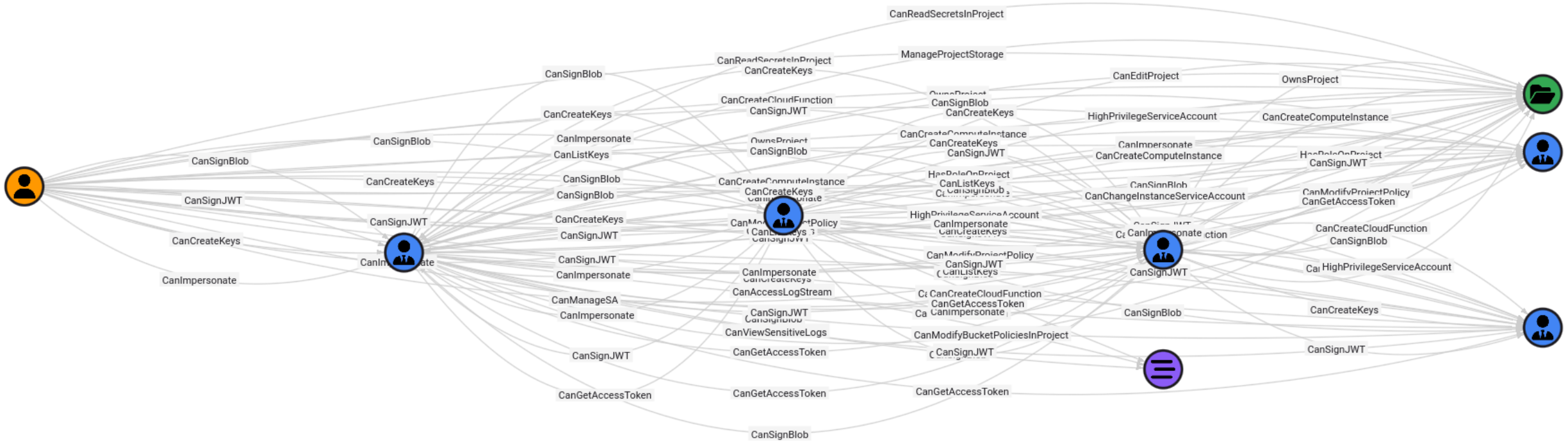
```
[*] Phase 5: Building Complete Attack Path Graph
[*] Edge Builder: Processing 1 projects, 1 IAM policies
[+] Built 27 IAM binding edges
[+] Built 9 service account edges (privilege analysis: IAM-based)
[+] Built 3 resource ownership edges
[+] Built 137 privilege escalation edges
[+] Edge Builder: Created 176 attack relationship edges
[+] Edge validation prevented invalid edges from being created
[+] Built 1 service account permission edges
[+] Built 0 folder relationship edges
[*] Built 199 total attack relationships
[*] Including 1 logging-specific edges
```

Phase 6: BloodHound export & upload

- Conversion of collected results in JSON.
- Credits to "Rémi GASCOU (Podalirius)" for his bhopengraph python library.

```
[DEBUG] Edges added: 139/167 (skipped 28 invalid edges)
[DEBUG] ✅ Clean export completed - no source_kind metadata
[+] ✅ FINAL RESULT: 24 nodes, 139 edges
[+] File: ./output/firebase_gcp-bhopgraph.json
[+] 🎯 GCP ATTACK SURFACE WITH NEW FEATURES COMPLETE

═══════════════════════════════════════════════════════════════════════
🎯 GCP ANALYSIS COMPLETE - LIMITED PERMISSIONS DETECTED
═══════════════════════════════════════════════════════════════════════

📊 RESOURCE INVENTORY:
   Projects: 1
   Service Accounts: 5
   Storage Buckets: 3
   Secrets: 0
   Compute Instances: 0
   BigQuery Datasets: 0
   GKE Clusters: 0
   Logging Resources: 5
     - Log Sinks: 3
     - Log Buckets: 2
     - Log Metrics: 0
   Users: 0
   Groups: 0
   IAM Bindings: 20
   Folders: 0

⚠ UNAVAILABLE APIS:
   GKE Clusters
   Enable APIs or use higher-privilege account for comprehensive analysis

🔗 ATTACK GRAPH:
   Total BloodHound Attack Edges: 208
   Logging Attack Edges: 1
   🚨 CRITICAL Attack Paths: 61

📁 OUTPUT:
   BloodHound JSON: ./output/firebase_gcp-bhopgraph.json
```

Upload to bloodhound



Node icons used in GCP-Hound

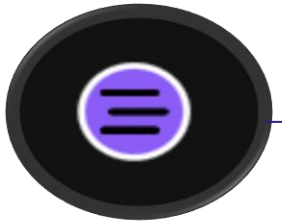
GCPUser

GCPServiceAccount

GCPBucket

GCPProject

GCPLogBucket

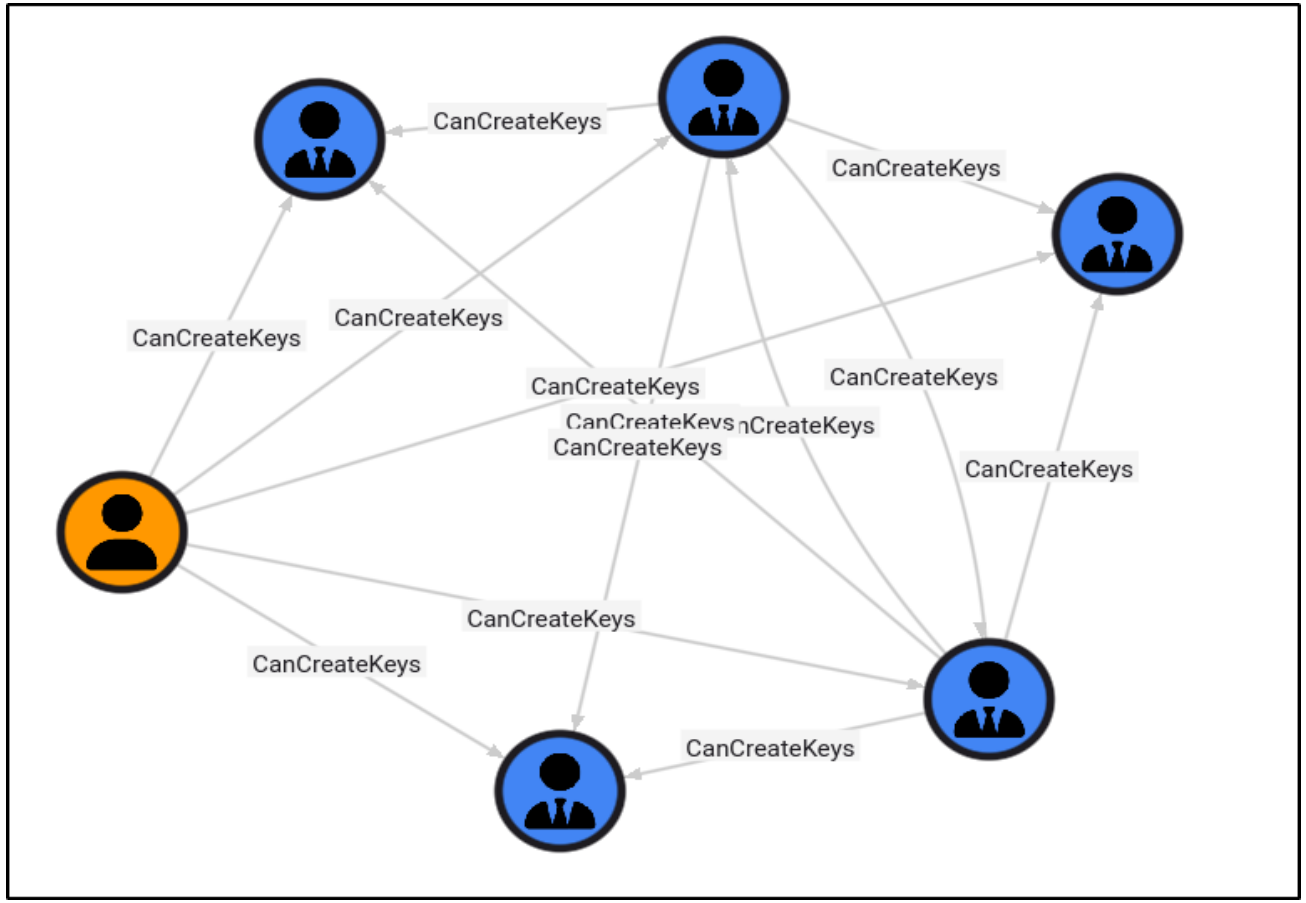


GCPLogSink

GCPGoogleManagedSA

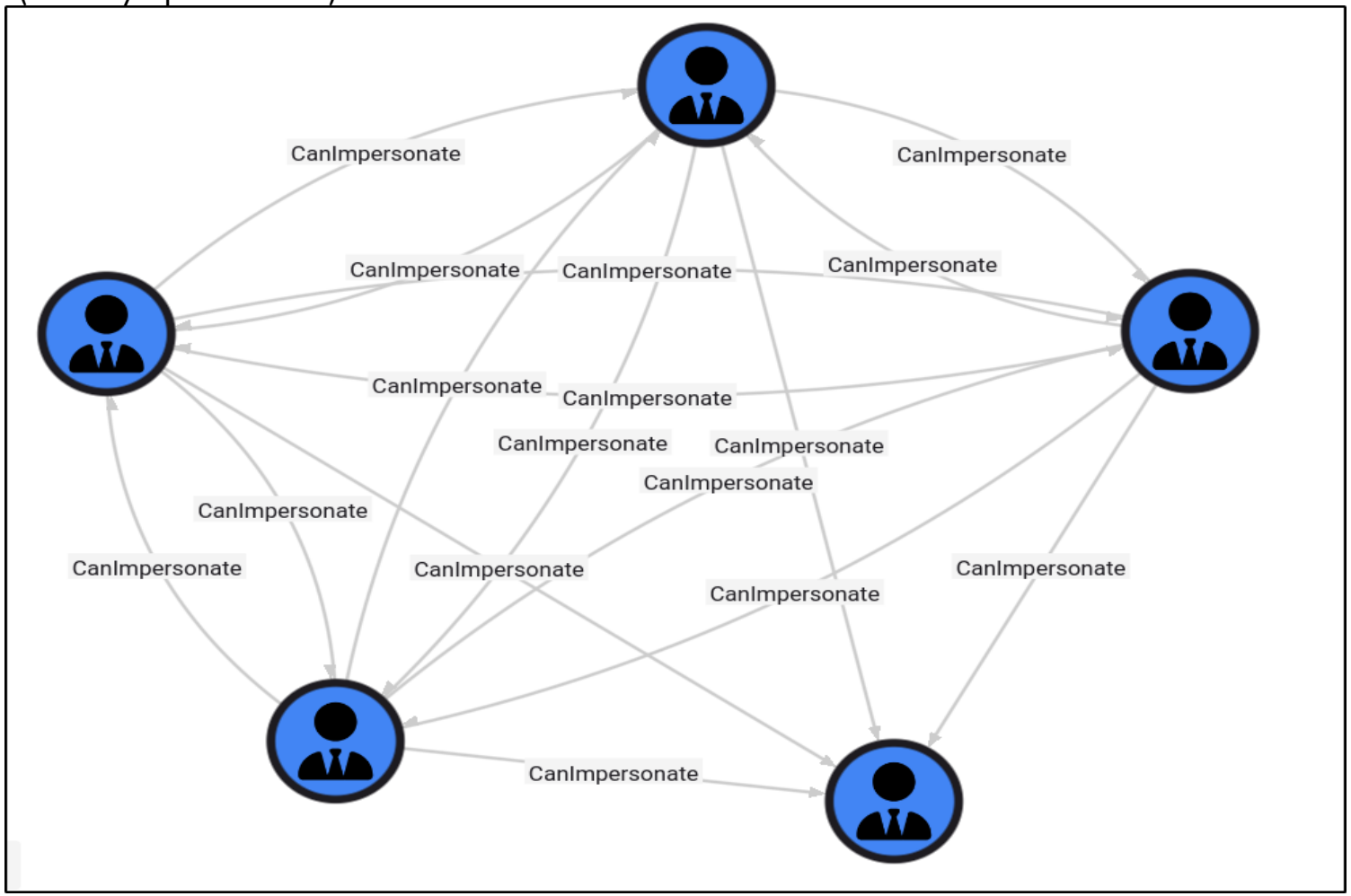
Edges used in GCP-Hound

CanCreateKeys - This is the permission *iam.serviceAccountKeys.create* (for service account keys) or *apikeys.keys.create* (for API keys). If an object has this permission over another service account then we can create a new key pair for that service account. This permission would allow us to create downloadable JSON or P12 file containing a private key that can be used to authenticate as that service account from anywhere.



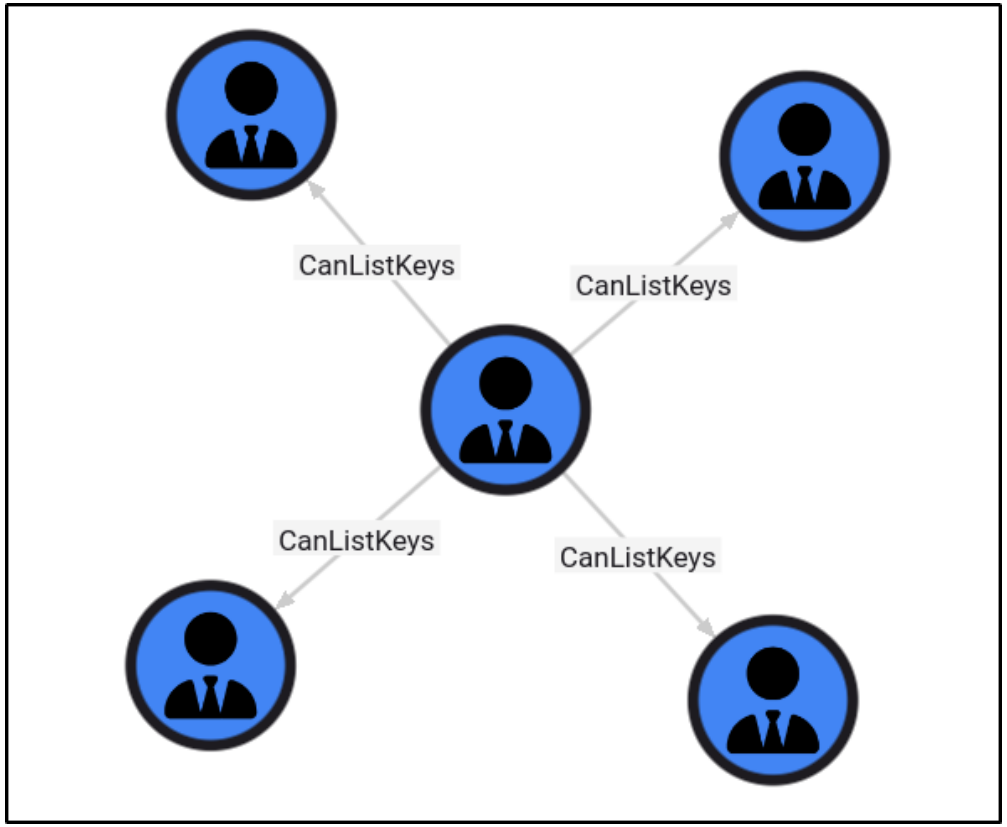
Edges used in GCP-Hound

CanImpersonate - This refers to having the *iam.serviceAccounts.getAccessToken* permission.
We can generate short-lived OAuth 2.0 access tokens for a service account without needing its key file. We effectively "become" that service account for a limited time (usually up to 1 hour).



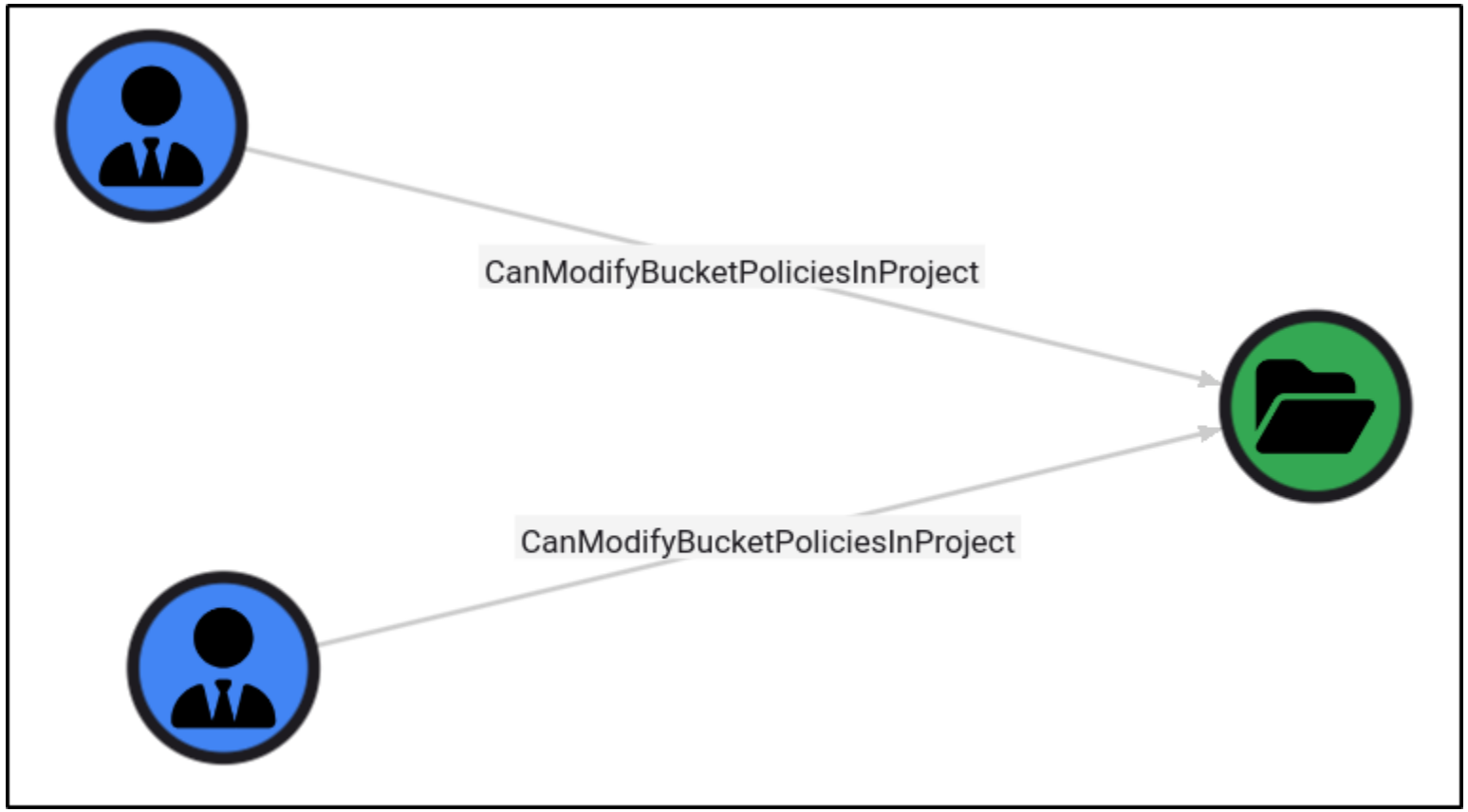
Edges used in GCP-Hound

CanListKeys - This refers to having the permission *iam.serviceAccountKeys.list*.
We can see all the keys associated with a service account, including when they were created, expiration dates, and key types. This is part of reconnaissance. Before we can steal or create keys, when we need to know what exist.



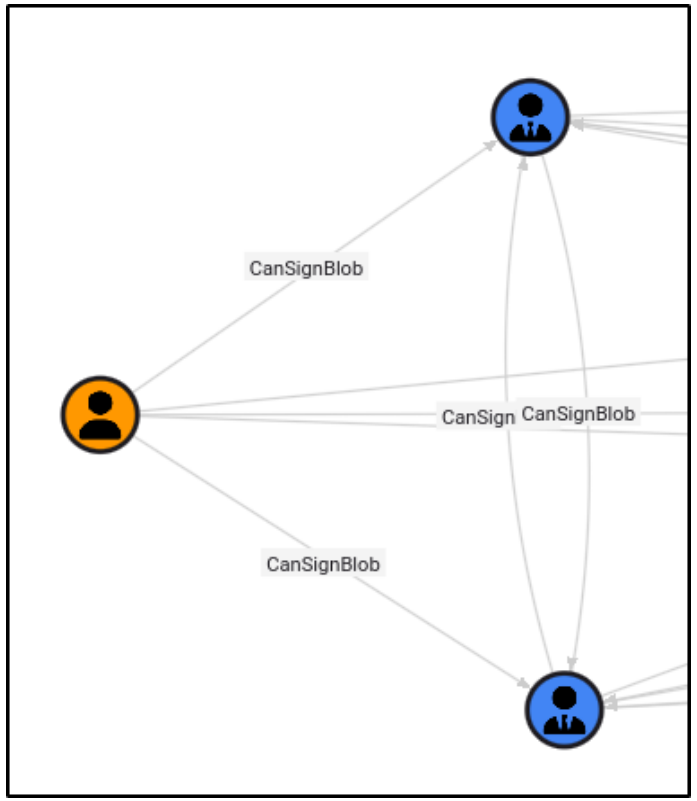
Edges used in GCP-Hound

CanModifyBucketPoliciesInProject - This is the permission of *storage.buckets.setIamPolicy* with the ability to apply it to any bucket in the project. This typically comes from *roles/storage.admin*.



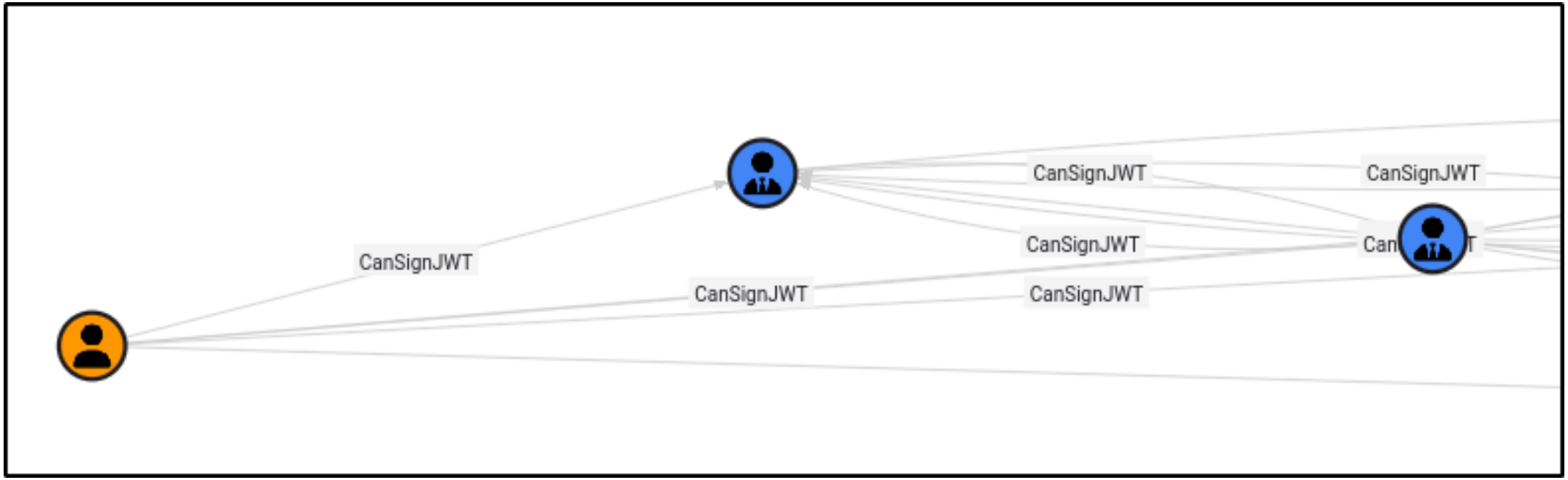
Edges used in GCP-Hound

CanSignBlob - The permission *iam.serviceAccounts.signBlob*. This lets us cryptographically sign arbitrary binary data using a service account's private key.
We can create digital signatures that prove the service account authorized something.
Ex: We can generate signed URLs for Cloud Storage that grant temporary access to private objects.



Edges used in GCP-Hound

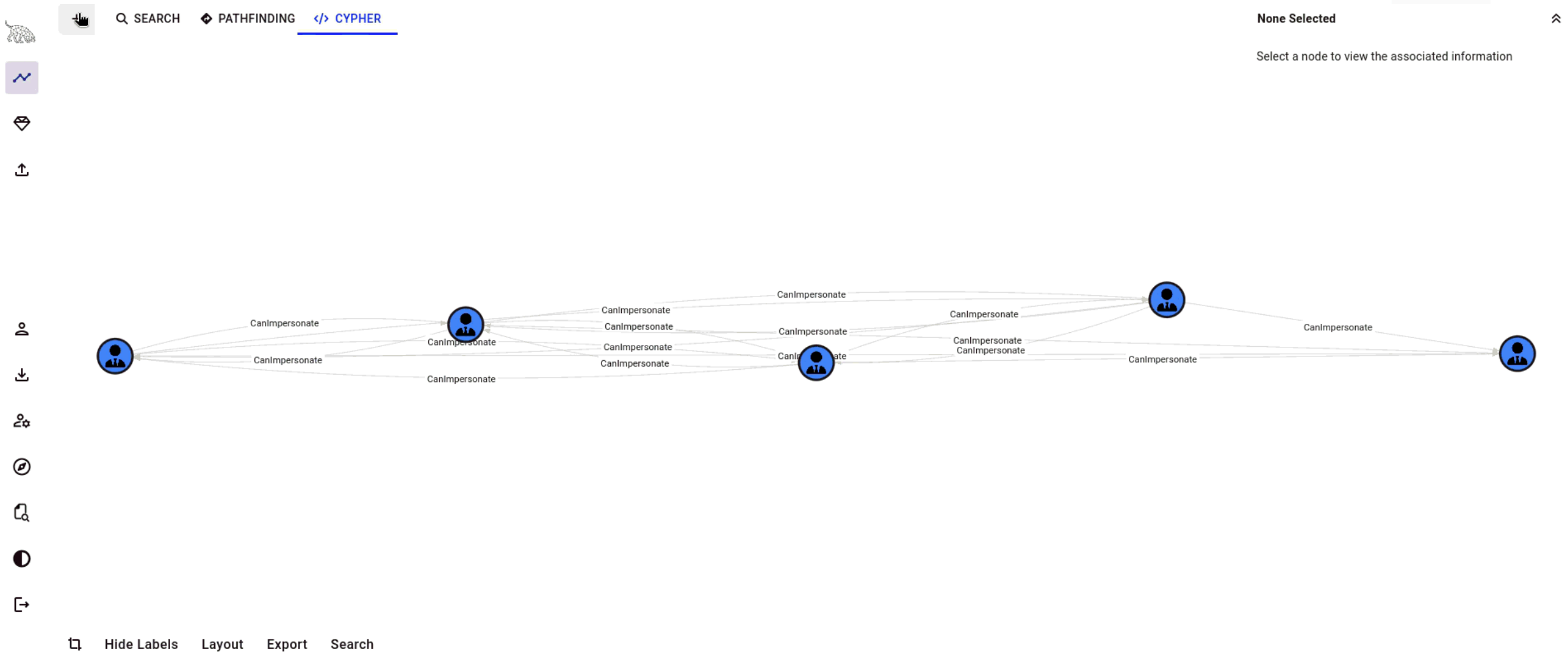
CanSignJWT - The permission refers to *iam.serviceAccounts.signJwt*. This is the JWT-specific version of signBlob.
We can create and sign properly formatted JSON Web Tokens that assert any identity claims we want, and then we can use that JWT to authenticate to any Google API that accepts JWTs



Manual enumeration vs GCP-Hound

```
└─$ gcloud iam service-accounts keys list --project=[redacted] --iam-account=[redacted]rviceaccount.com
KEY_ID                                    CREATED_AT            EXPIRES_AT            DISABLED  DISABLE_REASON  EXTENDED_STATUS
[redacted]7b5980eb88af8985a5e89243b  2025-05-16T14:47:25Z  9999-12-31T23:59:59Z
[redacted]e354c9dac7c5db640f2d3b176  2025-06-06T12:48:58Z  9999-12-31T23:59:59Z
[redacted]3f2b0a45a6f14700523069898  2025-06-06T11:41:30Z  2027-06-26T16:19:07Z

┌──(kali㉿kali)-[/opt/clients/google-keys-papouille]
└─$ gcloud iam service-accounts keys list --project=[redacted] --iam-account=[redacted]erviceaccount.com
KEY_ID                                    CREATED_AT            EXPIRES_AT            DISABLED  DISABLE_REASON  EXTENDED_STATUS
[redacted]524a58fb06a58fdce7a83bf1bf66ccb  2025-06-11T08:45:48Z  9999-12-31T23:59:59Z
[redacted]cc31afc81dbaf5ae707e2fefa31ced  2025-06-11T08:45:33Z  2027-06-25T00:35:02Z

┌──(kali㉿kali)-[/opt/clients/google-keys-papouille]
└─$ gcloud iam service-accounts keys list --project=[redacted] --iam-account=[redacted]am.gserviceaccount.com
KEY_ID                                    CREATED_AT            EXPIRES_AT            DISABLED  DISABLE_REASON  EXTENDED_STATUS
[redacted]38f764de56524bbe2ef175f7bcc585c  2025-06-11T08:44:51Z  2027-07-08T22:50:07Z

┌──(kali㉿kali)-[/opt/clients/google-keys-papouille]
└─$ gcloud iam service-accounts keys list --project=[redacted] --iam-account=[redacted]rviceaccount.com
KEY_ID                                    CREATED_AT            EXPIRES_AT            DISABLED  DISABLE_REASON  EXTENDED_STATUS
[redacted]def664034bd874220fdedf9b19f9  2024-10-16T09:00:54Z  9999-12-31T23:59:59Z
[redacted]513c73b4713bfb82ff4572635d97  2025-06-06T11:41:30Z  2027-06-10T23:56:31Z

┌──(kali㉿kali)-[/opt/clients/google-keys-papouille]
└─$ gcloud iam service-accounts keys list --project=[redacted] --iam-account=[redacted]rviceaccount.com
KEY_ID                                    CREATED_AT            EXPIRES_AT            DISABLED  DISABLE_REASON  EXTENDED_STATUS
[redacted]221cfa33fd6abbf6ad1c4e92c  2024-04-22T10:13:28Z  9999-12-31T23:59:59Z
[redacted]ee6c30095b07bdc77a4c8fd29  2024-07-02T14:20:56Z  9999-12-31T23:59:59Z
[redacted]2d015762b5c888e63cf1ba46d  2025-06-06T11:41:30Z  2027-07-02T16:08:49Z

┌──(kali㉿kali)-[/opt/clients/google-keys-papouille]
└─$ gcloud iam service-accounts keys list --project=[redacted] --iam-account=[redacted]iceaccount.com
KEY_ID                                    CREATED_AT            EXPIRES_AT            DISABLED  DISABLE_REASON  EXTENDED_STATUS
[redacted]ebe7cb57822f0a3bae0557f8  2024-06-20T12:07:35Z  9999-12-31T23:59:59Z
[redacted]47aefe509e57a82bfbb4457b  2024-10-14T10:29:20Z  9999-12-31T23:59:59Z
[redacted]f36650596e0def9b63272efa  2024-10-15T12:45:40Z  9999-12-31T23:59:59Z
```

Manual enumeration vs GCP-Hound



Demo

```
python3 gcp-hound.py -v
```

Future Plans

- **Community Engagement:** Welcoming contributions, feedback, and collaborative development.
- **Feature Development:** Expanding collector capabilities and improving edge relationship precision.

Shout out 📢

Rhino Security labs - https://github.com/RhinoSecurityLabs/GCP-IAM-Privilege-Escalation/tree/master

Garret Foster (unsigned_sh0rt): https://github.com/garrettfoster13/

Mathieu (Scoubi): https://www.linkedin.com/in/mathieusaulnier/

Thank you

GitHub: https://github.com/F41zK4r1m

LinkedIn: https://linkedin.com/in/faiz-karim-8421bb195

Mapping Shadows in the Cloud: Exploring Google Cloud Attack Paths with GCP-Hound and OpenGraph

SPECTEROPS

SO·CON

2026